



Functional Coverage

Introduction

Functional verification plays a critical role in ensuring the correctness and reliability of modern digital designs. With increasing design complexity, constraint-random testing combined with functional coverage has become a dominant methodology for validating design behavior under a wide range of scenarios.

This report presents the verification of a black-box register control module using a transaction-based SystemVerilog testbench. The verification environment applies randomized stimulus and functional coverage to evaluate the effectiveness of the testbench and to identify potential functional bugs in the Design Under Test (DUT). The goal of this work is to analyze DUT behavior, measure coverage completeness, and report any detected mismatches or protocol violations.

Assignment Overview

This assignment focuses on applying functional coverage techniques as part of a constraint-random verification flow to validate the correctness of a black-box digital design. The provided Design Under Test (DUT) is a register control module with memory read/write capabilities and an additional accumulate feature.

The DUT supports both accumulate-add and accumulate-multiply operations, controlled through dedicated control signals. The objective of the assignment was to design a complete SystemVerilog testbench that verifies the DUT behavior using randomized transactions while measuring verification completeness through functional coverage.

The verification task required:

- Building a transaction-based testbench architecture including generator, driver, monitor, checker, and coverage components.
- Defining multiple functional coverage points targeting address patterns, write/read operations, data values, accumulate modes, and function selection.
- Creating cross-coverage items to evaluate complex stimulus combinations.
- Executing verification in multiple phases, including reset validation, randomized operation, and final state checking.
- Ensuring correctness of the DUT outputs and protocol signals such as the ready signal.

The ultimate goal was to achieve 100% functional coverage using pseudo-random stimulus while identifying potential functional bugs in the DUT.



Verification Environment

A transaction-based SystemVerilog verification environment was developed to validate the functionality of the DUT. The testbench consists of the following components:

- Generator: Creates randomized transactions for address, control, and data fields.
- Driver: Applies generated transactions to the DUT interface.
- Monitor: Observes DUT inputs and outputs and reconstructs transactions.
- Scoreboard/Checker: Compares observed DUT behavior against expected results.
- Coverage Component: Collects functional coverage based on monitored transactions.

The verification environment operates continuously during simulation, monitoring DUT responses and reporting mismatches or protocol violations.

Functional Coverage Model

Functional coverage was implemented using SystemVerilog covergroups to evaluate the effectiveness of the randomized stimulus. Coverage points were defined to track both individual signal behavior and complex combinations of DUT activity.

The following coverpoints were implemented:

- Address coverage (addr_cp) to target boundary values, one-hot patterns, alternating bit patterns, and mid-range addresses.
- Write enable coverage (wr_cp) to distinguish read and write transactions.
- Accumulate mode coverage (acc_cp) and function selection coverage (func_cp).
- Write data coverage (wdata_cp), sampled during write operations.
- Read data coverage (rdata_cp), sampled during read operations.

Cross coverage was defined to analyze interactions between address, control signals, data values, and accumulate modes, ensuring complex operational scenarios were exercised during simulation.



Simulation Results

The verification environment was executed using constrained-random stimulus across multiple simulation cycles. Functional coverage data was collected and analyzed using the DVE coverage viewer.

The majority of defined coverpoints reached 100% coverage, indicating effective random stimulus generation. The overall functional coverage achieved was 85.71%, as shown in the DVE coverage report.

Simulation was performed using constrained-random stimulus without directed test cases.

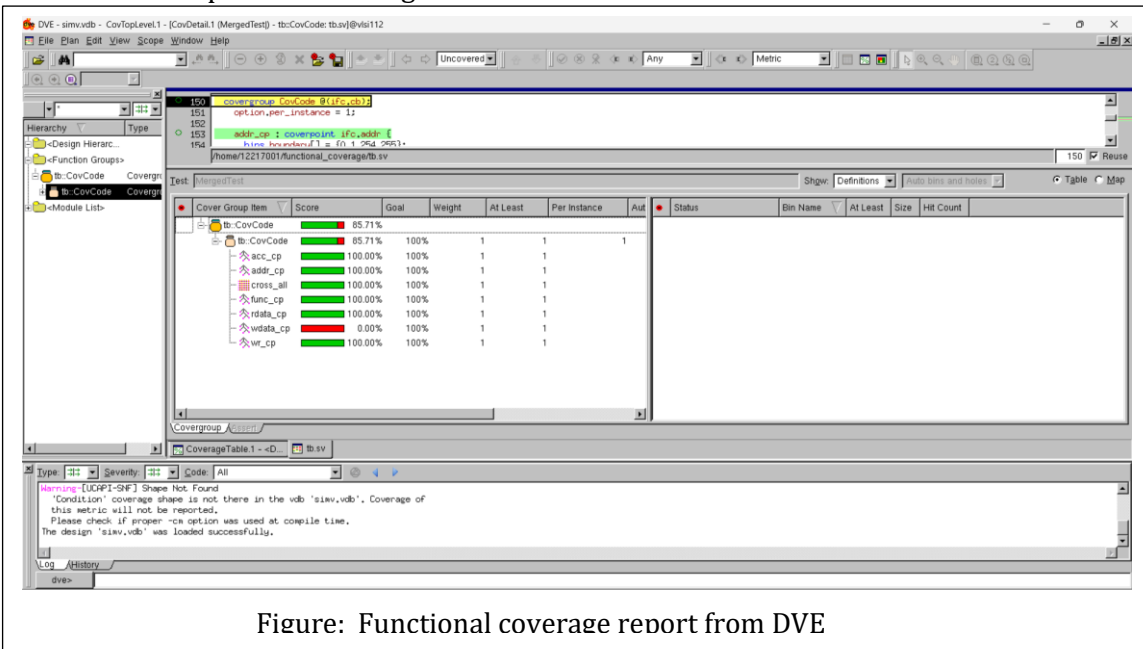


Figure: Functional coverage report from DVE

Bug Analysis: READY Signal Handshake Issue

Description of the Bug

During simulation, repeated error messages were reported indicating unexpected behavior of the ready signal. The simulator output repeatedly displayed messages of the form:

READY ERROR at time ...

These errors were observed consistently throughout the simulation, indicating a systematic issue rather than a random or transient condition.



Expected Behavior

The verification environment assumes that the ready signal indicates the successful completion of a read or write transaction. After a transaction is initiated, the ready signal is expected to

remain asserted long enough to be sampled by the checker, confirming that the operation has completed correctly.

Observed Behavior

Simulation results showed that the ready signal is asserted only while the sel signal is high and is deasserted immediately afterward. As a result, when the checker evaluates the transaction outcome, the ready signal is often already low, even though the transaction was correctly initiated. This behavior consistently triggered READY ERROR messages during simulation.

Why This Is a Functional Bug

This behavior represents a handshake protocol mismatch between the expected and actual behavior of the DUT. External logic relying on the ready signal may fail to detect transaction completion, leading to unreliable synchronization between system components. The issue was reproduced consistently under constrained-random testing, which confirms that it is a functional bug in the DUT rather than a testbench or simulation artifact.

Impact on Functional Coverage

The detected READY signal handshake issue had a direct impact on the functional coverage results. Although most coverpoints achieved full coverage, the wdata_cp coverpoint remained uncovered, resulting in an overall functional coverage of 85.71%.

This occurred because the wdata_cp coverpoint is sampled only during valid write operations. Due to the READY signal behavior, write transactions were not consistently completed, which prevented write data values from being sampled by the coverage model. As a result, the uncovered coverage bins are a direct consequence of the detected DUT bug rather than insufficient stimulus generation.



Conclusion

This report presented the functional verification of a black-box register control module using a transaction-based SystemVerilog testbench and constrained-random stimulus. Functional coverage was used to evaluate the completeness of the verification process and to assess how effectively the randomized transactions exercised the DUT behavior.

The verification environment successfully identified a READY signal handshake bug that caused protocol mismatches during transaction completion. This issue was consistently detected during simulation and directly impacted functional coverage results, preventing certain write-related coverage bins from being exercised.

Overall, the results demonstrate the effectiveness of functional coverage and constraint-random verification techniques in uncovering design issues that may not be detected through directed testing alone.